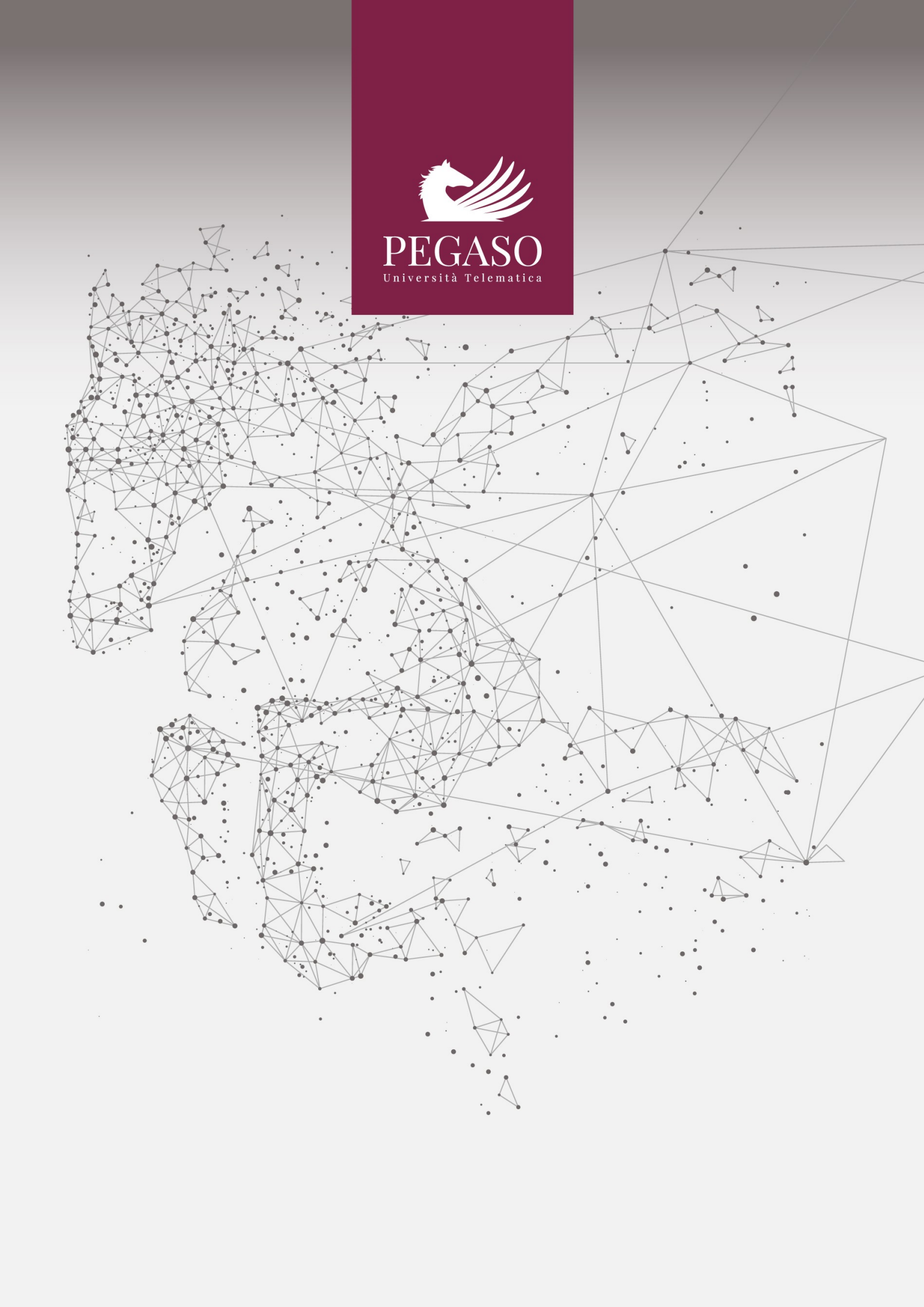

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE 3

2. ANALISI DEL PROBLEMA 4

3. PROBLEM SOLVING 7

3.1 FUNZIONE LANCIA () 7
3.2 FUNZIONE LANCIA_DADI (N_DADI, N_LANCI) 7
3.3 FUNZIONE SESSIONE_LANCI(N_RIPETIZIONI, N_DADI,N_LANCI) 8
3.4 FUNZIONE MEDIA_LANCI(TOTALE, NUMERO) 8
3.5 FUNZIONI MAIN () 9

4. FORMALIZZAZIONE DELLA SOLUZIONE 10

4.1 FUNZIONE LANCIA() 10
4.2 FUNZIONE LANCIA_DADI(N_DADI,N_LANCI) 10
4.3 FUNZIONE SESSIONE_LANCI(N_RIPETIZIONI, N_DADI,N_LANCI) 11
4.4 FUNZIONE MEDIA_LANCI(TOTALE, NUMERO) 12
4.5 FUNZIONI MAIN () 12
4.6 FUNZIONE VISUALIZZA_MENU() 15
4.7 ESEMPIO DI FUNZIONAMENTO DELL'INTERO PROGRAMMA 16

BIBLIOGRAFIA 17

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si presenta un caso di studio al fine di illustrare l'intero processo di scrittura del codice a partire dall'analisi del problema, passando per il problem solving, per arrivare alla formalizzazione della soluzione. Il caso di studio in esame è il simulatore di dadi molto importante per lo studio della probabilità e della statistica.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Analisi del problema

Il **caso di studio** che si propone è relativo ad un **simulatore di dadi** in Python. L'obiettivo di questa lezione è comprendere l'intero processo che parte dal problema e arriva alla formalizzazione della soluzione attraverso un programma.

Secondo un **approccio top-down** si analizza il problema e si individuano dei **sottoproblemi** da risolvere. Nel simulatore di dadi è possibile individuare i seguenti sottoproblemi:

1. simulazione del lancio di un dado;
2. scelta dei parametri per il lancio dei dadi (es. numero di dadi, numero di lanci);
3. operazioni ripetute di lancio di dadi;
4. calcolo di statistiche a seguito del lancio di dadi.

Una volta individuati i sottoproblemi è possibile pensare alle **funzioni** che li risolvono. In prima istanza, non è importante pensare al **contenuto della funzione**, ma solo alla sua **funzionalità** (**concetto di black-box**).

È molto utile pensare anche alla **funzione main**() che, solitamente, funge da **gestore delle funzioni** chiamandole in modo opportuno sulla base degli input ricevuti dall'utente.

Nel simulatore di dadi, quindi, potremmo prevedere le seguenti funzioni relative ai sottoproblemi individuati precedentemente:

1. **lancia()** per la simulazione del lancio di un dado;
2. **lancia_dadi(n_dadi,n_lanci)** per effettuare i lanci di uno o più dadi per *n_lanci* volte;
3. **sessione_lanci(n_ripetizioni, n_dadi,n_lanci)** per effettuare operazioni ripetute di lancio di dadi;
4. **media_lanci(totale, numero)** per il calcolo di statistiche a seguito del lancio di dadi.

Le funzioni precedentemente individuate possono essere schematizzate come mostrato in Figura 1 in base alle relazioni tra le funzioni stesse (funzioni invocanti e funzioni chiamate).

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

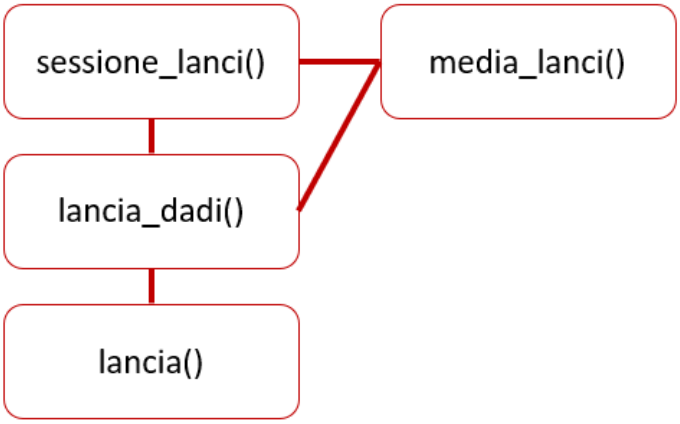


Figura 1. Schema delle funzioni individuate e delle relazioni tra di esse

Per poter richiamare in modo opportuno le funzioni, come anticipato, servirà un gestore.

Solitamente per gestire la chiamata delle funzioni sulla base di quanto scelto dall'utente, si utilizza un **menu**. Un menu è un **elenco di opzioni** tra le quali l'utente può scegliere per effettuare un'operazione specifica.

La gestione del menu solitamente avviene nella funzione **main**() attraverso una **struttura iterativa e una struttura decisionale (catena di if-elif-else).**

Il passo successivo nell'analisi del problema è, dunque, elencare le opzioni che si vogliono offrire all'utente del simulatore di dadi. Le opzioni non sono necessariamente i sottoproblemi individuati, ma, ad ogni modo, dipendono da essi.

Le opzioni potrebbero, dunque, essere le seguenti:

1. lanciare x dadi per y volte;
2. effettuare una sessione di z lanci di x dadi per y volte;
3. mostrare la media degli ultimi lanci ripetuti o dell'ultima sessione di lanci;
4. terminare il simulatore e uscire dal programma.

Riprendendo lo schema delle funzioni precedentemente analizzato è possibile aggiornarlo con la gestione del menu della funzione main() come mostrato in Figura 2.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

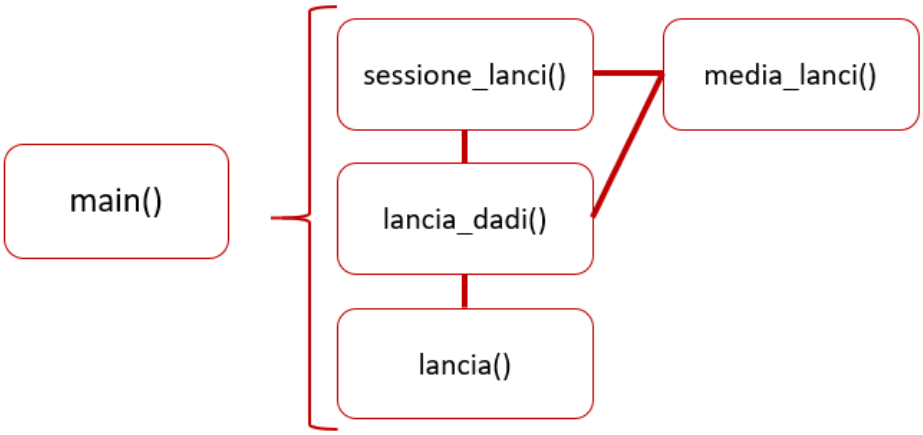


Figura 2. Schema delle funzioni individuate e delle relazioni tra di esse inclusa la funzione main

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Problem solving

A seguito dell'analisi del problema e dell'individuazione delle funzioni e delle opzioni da proporre all'utente del programma si passa alla **fase di problem solving**.

In questa fase sarà necessario capire come risolvere il problema prima della successiva formalizzazione della soluzione con il coding in Python.

Analizziamo, dunque, per ciascuna delle funzioni individuate nell'analisi del problema:

- Input (inteso come parametri passati come argomento);
- Output (inteso come valore restituito o stampa);
- Elaborazione.

3.1 Funzione lancia ()

Input: nessuno.

Output: un numero da 1 a 6 scelto in modo casuale per simulare il lancio di un dado.

Elaborazione: è necessario individuare in Python un **modulo per i numeri casuali**. Il modulo **random** in Python include funzioni per lavorare con numeri casuali. In particolare, la **funzione randint**(a,b) del modulo random genera e restituisce un numero intero casuale nell'intervallo [a,b].

3.2 Funzione lancia_dadi (n_dadi, n_lanci)

Input: numero dei dadi (*n_dadi*), numero di lanci da effettuare per ciascun dado (*n_lanci*).

Output: stampa di *n_dadi* numeri da 1 a 6 per *n_lanci* volte. Restituisce, inoltre, la somma dei numeri estratti.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Elaborazione: è necessario richiamare la funzione lancia() per *n_dadi* per *n_lanci* volte. Sarà possibile utilizzare due cicli annidati:

1. uno esterno che itera sui lanci;
2. uno più interno che itera sui dadi.

3.3 Funzione sessione_lanci(n_ripetizioni, n_dadi,n_lanci)

Input: numero dei dadi (*n_dadi*), numero di lanci da effettuare per ciascun dado (*n_lanci*), numero delle sessioni di lancio da effettuare (*n_ripetizioni*).

Output: stampa di *n_dadi* numeri da 1 a 6 per *n_lanci* volte per *n_ripetizioni* sessioni.

Restituisce, inoltre, la somma di tutti i numeri estratti.

Elaborazione: è necessario richiamare la funzione lancia() per *n_dadi* per *n_lanci* per *n_sessioni*. Sarà possibile utilizzare tre cicli annidati:

1. uno più esterno che itera sulle sessioni;
2. uno esterno che itera sui lanci;
3. uno più interno che itera sui dadi.

Ad ogni modo, i due cicli più interni potranno essere sostituiti da una chiamata alla funzione lancia_dadi() che già prevede il lancio di *n_dadi* dadi per *n_lanci* volte.

3.4 Funzione media_lanci(totale, numero)

Input: *totale* (somma) di tutti gli ultimi numeri estratti e *numero* dei lanci corrispondenti.

Output: restituisce la media dei numeri estratti.

Elaborazione: è necessario dividere (divisione in virgola mobile) il totale e il numero e, successivamente, restituire tale valore.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3.5 Funzioni main ()

Input: nessuno.

Output: stampa valori restituiti dalle funzioni.

Elaborazione: è necessario implementare il menu delle opzioni per l'utente e richiamare opportunamente le funzioni viste in precedenza.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Formalizzazione della soluzione

Si procede ora a formalizzare il codice sorgente di ciascuna funzione.

4.1 Funzione lancia()

Come anticipato, per l'estrazione di un numero casuale tra 1 e 6 per simulare il lancio di un dado è necessaria la **funzione randint**() (intero casuale) del **modulo random**. Sarà necessario, quindi, importare il modulo e successivamente definire la funzione che ritornerà il risultato della chiamata alla funzione randint() nell'intervallo [1-6]. Segue il codice sorgente della funzione:

```
import random

def lancia():
    return random.randint(1,6)
```

4.2 Funzione lancia_dadi(n_dadi,n_lanci)

La **funzione lancia_dadi()** simula il lancio di *n_dadi* dadi per un totale di *n_lanci* lanci. I parametri *n_dadi* e *n_lanci* vengono passati alla funzione come argomenti. La variabile *acc,* che viene inizializzata a zero, fungerà da accumulatore per il calcolo del totale dei numeri estratti che ricordiamo sarà pari a *n_dadi*n_lanci*.

La funzione utilizza due strutture iterative annidate:

1. la prima itererà tante volte quanti sono i lanci. Proprio per questo motivo si richiama la funzione print() che visualizzerà a schermo il numero del lancio in questione. Si utilizza i+1 poiché si ricorda che la funzione range parte da zero;
2. la seconda itererà tante volte quanti sono i dadi. In particolare, la struttura iterativa annidata permette di iterare sull'indice *j* nell'intero intervallo [0, *n_dadi*-1] per ogni valore dell'indice *i* del ciclo più esterno.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La **funzione lancia()** verrà, dunque, chiamata *n_dadi*n_lanci* volte e a schermo verranno visualizzati volta per volta i numeri estratti. Prima di terminare il ciclo interno, il valore estratto viene "accumulato" nella variabile accumulatore *acc*. La chiamata alla funzione print() senza argomenti servirà solo a separare visivamente a schermo i lanci. Al termine dell'esecuzione dei due cicli annidati, la funzione restituirà il totale dei numeri estratti. Segue il codice sorgente della funzione:

```
def lancia_dadi(n_dadi,n_lanci):
    acc = 0
    for i in range(n_lanci):
        print("Lancio n."+str(i+1))
        for j in range(n_dadi):
            l = lancia()
            print("Dado n."+str(j+1)+"="+str(l))
            acc = acc+l
        print()
    return acc
```

4.3 Funzione sessione_lanci(n_ripetizioni, n_dadi,n_lanci)

La funzione **sessione_lancia**() simula *n_ripetizioni* sessioni di lancio di *n_dadi* dadi per *n_lanci* lanci. I parametri *n_ripetizioni*, *n_dadi* e *n_lanci* vengono passati alla funzione come argomenti. La variabile *acc*, che viene inizializzata a zero, fungerà da accumulatore per il calcolo del totale dei numeri estratti che ricordiamo sarà pari a *n_ripetizioni*n_dadi*n_lanci*. In questo caso, si utilizzerà una sola struttura iterativa che itererà sulle sessioni: le iterazioni sui lanci e sui dadi saranno già comprese nella chiamata alla **funzione lancia_dadi**(). Si noti come il codice organizzato in funzioni è più leggibile: se non avessimo utilizzato due funzioni, avremmo dovuto gestire tre strutture iterative annidate.

Nell'accumulatore *acc*, quindi, verranno accumulati i valori estratti di ogni sessione. Dopo ciascuna sessione, si richiama la funzione print() che, per come è scritta, equivale ad andare a capo due volte al fine di separare visivamente a schermo le sessioni tra loro. Segue il codice sorgente della funzione:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
def sessione_lanci(n_ripetizioni, n_dadi,n_lanci):
    acc = 0
    for i in range(n_ripetizioni):
        print("Sessione n.",i+1)
        acc= acc+ lancia_dadi(n_dadi,n_lanci)
        print("\n")
    return acc
```

4.4 Funzione media_lanci(totale, numero)

La funzione **media_lanci**() calcola e restituisce la media espressa come numero in virgola mobile frutto della divisione tra il totale dei numeri estratti e la loro numerosità. Si noti la necessità di effettuare un casting a float di uno dei due valori interi passati alla funzione (in questo caso totale) per effettuare una divisione non intera. Segue il codice sorgente della funzione:

```
def media_lanci(totale, numero):
    return float(totale)/numero
```

4.5 Funzioni main ()

Per poter realizzare un menu di opzioni per l'utente si individua preliminarmente il numero e un nome significativo per ciascuna opzione che si vuole offrire. Le opzioni individuate in precedenza sono:

1. lanciare x dadi per y volte – **LANCIO**;
2. effettuare una sessione di z lanci di x dadi per y volte – **SESSIONE**;
3. mostrare la media dell'ultimo lancio ripetuto o dell'ultima sessione di lanci – **MEDIA**;
4. terminare il simulatore e uscire dal programma – **ESCI**.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Le opzioni individuate come nome e numero diventano le seguenti costanti globali che vengono dichiarate al di fuori della funzione main():

```
LANCIO = 1
SESSIONE = 2
MEDIA = 3
ESCI = 4
```

Per evitare di appesantire il codice della funzione main() con una serie di chiamate alla funzione print() per mostrare tutte le opzioni possibili è consigliabile utilizzare un'altra funzione per la stampa delle indicazioni a schermo del menu **visualizza_menu**() (vedi paragrafo 4.6).

La variabile *acc* conterrà il totale dei numeri estratti nell'ultima serie o nell'ultima sessione di lanci, mentre la variabile *num_lanci_tot* servirà a tenere traccia del numero di lanci nell'ultima serie o nell'ultima sessione.

Per la gestione del menu, si utilizzerà una variabile numerica intera *scelta* per la selezione dell'opzione dell'utente. La variabile *scelta* viene inizializzata ad un valore al di fuori di quello logicamente valido (in questo caso 0 fuori dall'intervallo 1-4).

L'inizializzazione precedente permette di entrare in un ciclo while che itera finché la scelta dell'utente è diversa dall'opzione di uscita (nel nostro caso 4). A seguito dell'input da parte dell'utente:

- se la scelta è fuori dall'intervallo delle opzioni (nel nostro caso 1-4), si chiederà all'utente di ripetere la scelta;
- se la scelta è nell'intervallo, una struttura decisionale richiamerà le funzioni corrispondenti in modo opportuno.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il codice seguente imposta il gestore del menu:

```
def main():
    acc =0
    num_lanci_tot=0
    scelta = 0

    while scelta!=ESCI:
        visualizza_menu()
        scelta = int(input("Selezionare una delle opzioni (1-4):\t"))
```

La struttura decisionale all'interno del ciclo while, invece, permette di eseguire le richieste dell'utente. In base all'opzione (espressa come numero intero) decisa dall'utente, la funzione provvede a prendere in input i dati necessari per la simulazione e a chiamare la funzione corrispondente.

Per la prima opzione **"Lancio dei dadi"**, vengono richiesti il numero dei dadi (*num*) e il numero dei lanci (*lanci*) per, poi, effettuare la simulazione di una serie di lanci di dadi con la chiamata alla funzione **lancia_dadi**() passando i parametri *num* e *lanci*. La somma dei numeri estratti viene assegnata alla variabile *acc* ed alla variabile *num_lanci_tot* viene assegnato il risultato della moltiplicazione tra *num* e *lanci* (che rappresenta il numero totale dei numeri estratti).

Queste ultime operazioni servono nel caso in cui l'utente richieda di calcolare la media dell'ultima serie di lanci.

Per la seconda opzione **"Sessioni di lancio dei dadi"**, vengono richiesti il numero dei dadi (*num*), il numero dei lanci (*lanci*) e il numero di sessioni di lancio (*sessioni*) per, poi, effettuare la simulazione di sessioni di lanci di dadi con la chiamata alla funzione **sessione_lanci**() passando i parametri *sessioni*, *num* e *lanci*. La somma dei numeri estratti viene assegnata alla variabile *acc* e alla variabile *num_lanci_tot* viene assegnato il risultato della moltiplicazione tra *sessioni*, *num* e *lanci* (che rappresenta il numero totale dei numeri estratti). Queste ultime operazioni servono nel caso in cui l'utente richieda di calcolare la media delle ultime sessioni di lanci.

Per la terza opzione **"Media degli ultimi lanci"**, non viene richiesto nessun input. Si visualizza a schermo il risultato della chiamata alla funzione **media_lanci**() passando i parametri *acc* e *num_lanci_tot* avendo cura di riportare il risultato con due cifre decimali.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per la quarta ed ultima opzione **"Uscire dal programma"**, non viene richiesto nessun input. Si visualizza a schermo solo un messaggio per comunicare all'utente che il programma verrà terminato.

L'else finale serve per intercettare tutte le scelte al di fuori dell'intervallo [1-4] al fine di comunicare all'utente della scelta non valida e ripetere la scelta dell'opzione. Segue il codice sorgente della funzione:

```
if scelta == LANCIO:
    num = int(input("Inserire il numero di dadi: "))
    lanci = int(input("Inserire il numero di lanci per ciascun dado: "))
    print()
    num_lanci_tot=num*lanci
    acc = lancia_dadi(num,lanci)

elif scelta == SESSIONE:
    sessioni = int(input("Inserire il numero di sessioni di lancio: "))
    num = int(input("Inserire il numero di dadi: "))
    lanci = int(input("Inserire il numero di lanci per ciascun dado: "))
    print()
    num_lanci_tot=num*lanci*sessioni
    acc = sessione_lanci(sessioni,num,lanci)

elif scelta == MEDIA:
    print("La media degli ultimi lanci è", format(media_lanci(acc, num_lanci_tot),".2f"))
    print()

elif scelta == ESCI:
    print("Esco dal programma...")

else:
    print("Errore, scelta non valida")
```

4.6 Funzione visualizza_menu()

La funzione **visualizza_menu**() ha il solo scopo di stampare a schermo le opzioni del menu.

Serve principalmente a ridurre il numero di linee di codice all'interno della funzione main() per aumentarne la leggibilità. Segue il codice sorgente della funzione:

```
def visualizza_menu():
    print("\n\tMENU")
    print("1. Lancio dei dadi")
    print("2. Sessioni di lancio di dadi")
    print("3. Media degli ultimi lanci")
    print("4  Uscire dal programma\n")
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4.7 Esempio di funzionamento dell'intero programma

Di seguito si riporta un esempio di esecuzione del programma. Per esigenze di spazio, si riporta un caso molto semplice di lancio di un dado per una sola volta e la successiva stampa della media dell'ultima serie di lanci (che nel caso mostrato conterrà solo un valore estratto pari a 1). Il lettore può sperimentare l'esecuzione del codice con parametri diversi e verificare che per grandi numeri di lanci, la media dei numeri estratti tende a 3.

```
        MENU
1. Lancio dei dadi
2. Sessioni di lancio di dadi
3. Media degli ultimi lanci
4  Uscire dal programma

Selezionare una delle opzioni (1-4):    1

Inserire il numero di dadi: 1
Inserire il numero di lanci per ciascun dado: 1

Lancio n.1
Dado n.1=1


        MENU
1. Lancio dei dadi
2. Sessioni di lancio di dadi
3. Media degli ultimi lanci
4  Uscire dal programma

Selezionare una delle opzioni (1-4):    3

La media degli ultimi lanci è 1.00
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Gaddis, T. (2016). Introduzione a Python. Ediz. mylab. Con espansione online. Pearson.
- Python (2021). https://www.python.org/

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).